

Univerzitet u Beogradu
Matematički fakultet

Analiza projekta otvorenog koda

ChatterBot

Seminarski rad iz predmeta **Verifikacija softvera**

Autor: Bogdan Tomić
Broj indeksa: 1040/2025
GitHub: <https://github.com/BogBogdan>
Tip rada: analiza projekta otvorenog koda (samostalni)

Beograd, 2026.

Sadržaj

1	Uvod	2
2	O projektu koji sam analizirao	2
2.1	Kako je biblioteka podeljena	2
2.2	Šta se dešava kada bot dobije poruku	2
3	Obim analize	3
4	Okruženje	3
5	Alati	4
5.1	Jedinični testovi	4
5.2	Integracioni testovi	4
5.3	Property-based testiranje	5
5.4	flake8	5
5.5	mypy	6
5.6	bandit	6
6	Pronađeni problemi	6
6.1	Parser datuma puca na običnim frazama	7
6.2	Zaštita od path traversal napada može da se zaobiđe	7
6.3	Bot ponavlja ono što je korisnik rekao	8
6.4	Izvezeni podaci ne mogu da se uvezu	8
7	Pokrivenost koda	8
8	Ograničenja	9
9	Zaključak	9
	Prilog: struktura repozitorijuma	9

1 Uvod

U ovom radu analiziram biblioteku otvorenog koda ChatterBot pomoću šest alata i tehnika za verifikaciju softvera. Zadatak je bio pronaći greške ili uska grla u programu, pri čemu se kod analiziranog projekta ne menja.

Projekat sam uključio u svoj repozitorijum kao git submodule zaključan na jedan određen komit, tako da svi rezultati mogu da se ponove. Sve što sam pisao — testovi, skripte za pokretanje alata i izveštaji — nalazi se u mom repozitorijumu, odvojeno od tuđeg koda.

Radio sam u četiri koraka. Prvo sam pustio alate koji čitaju kod bez pokretanja (flake8, mypy i bandit), jer su brzi i pokazuju gde ima sumnjivih mesta. Zatim sam pisao jedinične testove i uz njih merio pokrivenost koda, pa integracione testove koji rade nad pravom bazom, i na kraju property-based testove koji sami traže ulaz na kome program pukne.

Redosled se pokazao korisnim. Statički alati su me uputili na module `parsing.py` i `trainers.py`, a tamo su testovi zaista i našli greške. Ukupno sam pronašao deset problema, od kojih je jedan bezbednosne prirode i za njega sam napisao dokaz da može da se iskoristi.

2 O projektu koji sam analizirao

ChatterBot je Python biblioteka za pravljenje chat botova, objavljena 2014. godine. Ne koristi veliki jezički model. Umesto toga, bot pamti rečenice u bazi, a kada dobije novo pitanje, traži najbližnju zapamćenu rečenicu i vraća ono što je zapisano kao odgovor na nju.

Repozitorijum	https://github.com/gunthercox/ChatterBot
Grana	master
Heš komita	feafc81a00be8b8bb65622e4ddc89ebf7b3e1329
Verzija	1.2.13
Licenca	BSD-3-Clause
Jezik	Python 3.10–3.14
Veličina	oko 10.400 linija koda

Tabela 1: Podaci o analiziranom projektu

2.1 Kako je biblioteka podeljena

Kod je podeljen na sedam celina. Klasa `Statement` u `conversation.py` predstavlja jednu rečenicu i osnovni je podatak celog sistema. Skladište (`storage/sql_storage.py`) upisuje te rečenice u bazu i čita ih nazad, preko SQLAlchemy sloja. Tagger (`tagging.py` i `components.py`) koristi spaCy da od teksta napravi indeks po kome se kasnije pretražuje. Pretraga (`search.py`) pronalazi kandidate u bazi, a komparatori (`comparisons.py`) računaju koliko su dva teksta slična. Logic adapteri u folderu `logic/` odlučuju šta će bot odgovoriti, a treneri (`trainers.py`) pune bazu podacima.

2.2 Šta se dešava kada bot dobije poruku

Poziv `bot.get_response("Hello")` počinje u `chatterbot.py:162` i prolazi kroz sledeće korake:

```
ulazni tekst
-> Statement objekat                chatterbot.py:202
-> preprocesori (ciscenje beline)    chatterbot.py:217
-> search_text preko spaCy tagera     chatterbot.py:226
-> svaki logic adapter daje predlog i pouzdanost  chatterbot.py:271
-> pobeđuje predlog sa najvećom pouzdanoscu
-> upis ulaza i odgovora u bazu (ucenje)        chatterbot.py:262
odgovor
```

Jedan detalj je bitan za razumevanje kasnijih nalaza. Pretraga ne traži rečenicu koja liči na ono što je korisnik napisao, nego rečenicu koja je nekada bila izgovorena kao odgovor na nešto slično. Zbog toga svaka rečenica u bazi ima dva indeksa: jedan za sopstveni tekst i jedan za tekst na koji odgovara.

3 Obim analize

Analizirao sam jezgro biblioteke, odnosno put poruke od ulaza do odgovora. To je oko 2.100 izvršnih iskaza po merenju alata `coverage.py` i obuhvata module `chatterbot.py`, `preprocessors.py`, `comparisons.py`, `response_selection.py`, `conversation.py`, `utils.py`, `parsing.py`, `tagging.py`, `storage/sql_storage.py`, `trainers.py` i dva logic adaptera.

Neke delove sam namerno izostavio. Skladišta za Redis i MongoDB, kao i `vectorstores.py`, traže da na računaru bude pokrenut odgovarajući server. Django integracija traži konfigurisan Django projekat. Modul `languages.py` sam izostavio iz drugog razloga: to je tabela od 2.418 linija podataka o jezicima, bez ikakve logike, pa bi merenje pokrivenosti nad njim samo naduvalo procenat.

Statičke alate sam ipak pustio nad celim paketom, jer njihovo pokretanje ništa ne košta. Zbog toga se u njihovim rezultatima ponegde pojavljuju i moduli van obima.

4 Okruženje

Radio sam na Windows-u 11, sa Python-om 3.12.10 i spaCy modelom `en_core_web_sm` verzije 3.8.0. Verzije alata su `pytest` 9.0.3, `coverage` 7.14.0, `hypothesis` 6.145.1, `flake8` 7.3.0, `mypy` 1.19.0 i `bandit` 1.9.2. Sve verzije su zakucane u `requirements.txt`, jer bi inače brojke iz ovog izveštaja, na primer “mypy prijavljuje 46 grešaka”, prestale da važe sa novom verzijom alata.

Rezultati se ponavljaju ovako:

```
git clone --recurse-submodules <url-repozitorijuma>
cd 2026_Analysis_ChatterBot

python -m venv .venv
.venv/Scripts/activate          # Windows
source .venv/bin/activate       # Linux, macOS

bash scripts/priprema.sh
bash scripts/pokreni_sve.sh
```

U repozitorijumu postoji i konfiguracija za GitHub Actions, koja iste alate pokreće na Ubuntu serveru. Tako se vidi da rezultati ne zavise od toga što je nešto već instalirano na

mom računaru.

5 Alati

#	Alat	Šta radi	Rezultat
1	pytest, jedinični testovi, uz coverage.py	112 testova nad pojedinačnim funkcijama	112 prolazi, pokrivenost 30%
2	pytest, integracioni testovi	18 testova nad pravom bazom i spaCy modelom	18 prolazi, pokrivenost 47%
3	Hypothesis	sam generiše ulaze i traži kontraprimer	10 prolazi, 5 uzroka pada
4	flake8	provera stila i sitnih grešaka	3 prijave, odnosno 87 sa strožom granicom
5	mypy	provera tipova bez pokretanja koda	46 grešaka
6	bandit	traži poznate bezbednosne obrasce	6 nalaza, jedan visokog prioriteta

Tabela 2: Alati korišćeni u radu

Alat `coverage.py` se po pravilima kursa ne broji kao zaseban alat jer je deo podrške za testiranje. Alati koje nismo radili na vežbama su Hypothesis, mypy i bandit, a alat za proveru stila je samo jedan.

5.1 Jedinični testovi

Jedinični test proverava jednu funkciju u izolaciji, bez baze i bez mreže. Tamo gde funkcija traži nešto spolja, dajem joj lažni objekat. Na primer, funkcija `get_most_frequent_response` od skladišta koristi samo metodu `filter`, pa joj umesto prave baze prosleđujem klasu koja filtrira običnu listu:

```
class LazniStorage:
    def __init__(self, recenice):
        self.recenice = recenice

    def filter(self, **kwargs):
        trazeno = kwargs.get('in_response_to')
        return [r for r in self.recenice if r.in_response_to == trazeno]
```

Napisao sam 112 takvih testova u šest fajlova, za preprocesore, komparatore, klasu `Statement`, izbor odgovora, `utils.py` i matematički adapter. Uz njih merim pokrivenost, ali ne da bih dobio što veći procenat, nego da bih video koji delovi koda uopšte nisu dodirnuti.

5.2 Integracioni testovi

Ovde ništa nije zamenjeno lažnim objektima. Koristi se prava SQLite baza na disku, pravi spaCy model i pravi treneri, pa se testira kako komponente rade zajedno. Takvih

testova ima 18, u dva fajla. Prvi pokriva rad sa bazom, uključujući proveru da podaci prežive zatvaranje veze, a drugi ceo tok od treninga do odgovora, zajedno sa učenjem kroz razgovor i radom više adaptera.

Koliko ovaj sloj dodaje najbolje se vidi na modulu `trainers.py`: jedinični testovi ga uopšte ne dodiruju, a integracioni ga dižu na 44%. Nalazi 9 i 10 mogli su biti pronađeni samo ovako, jer nastaju tek kada se komponente spoje.

5.3 Property-based testiranje

Kod ovakvih testova se ne zadaje pojedinačan primer nego svojstvo koje mora da važi za svaki ulaz, a alat sam pravi ulaze i traži onaj koji ga obori. Jedno takvo svojstvo je da čišćenje već očišćenog teksta ne sme ništa da promeni:

```
@given(st.text())
def test_ciscenje_je_idempotentno(tekst):
    jednom = clean_whitespace(Statement(text=tekst)).text
    dvaput = clean_whitespace(Statement(text=jednom)).text

    assert jednom == dvaput
```

Kod modula `parsing.py` nasumičan tekst nije bio od koristi, jer takav tekst gotovo nikada ne liči na datum. Zato sam ulaze sastavljao od reči koje sam parser prepoznaje, kao što su `january`, `next`, `end of the` ili `31st`. Skripta `tools/hypothesis/nadji_kontraprimere.py` na pet hiljada takvih fraza nalazi pet različitih grešaka:

```
'last month last monday'      -> ValueError: month must be in 1..12
'30 31st february in'         -> ValueError: day is out of range for month
'in day 29 hours'             -> ValueError: hour must be in 0..23
'end of the week end of the of' -> TypeError: 'int' + 'timedelta'
'31 friday end of the friday'  -> TypeError: 'NoneType' + 'timedelta'
```

Skripta je podešena sa `derandomize=True`, pa svako pokretanje daje isti spisak. Bez toga bi se broj nalaza menjao između pokretanja i izveštaj ne bi bio ponovljiv.

5.4 flake8

Ovaj alat čita kod i traži kršenja stila, ali i sitne logičke propuste kao što su promenljive koje se nigde ne koriste. Pustio sam ga dva puta. Prvi put sa konfiguracijom samog projekta, da bih video da li projekat poštuje sopstveni standard, i drugi put sa strožom granicom od 100 kolona, da bih video šta bi prijavio uobičajeni PEP 8.

Sa projektnom konfiguracijom dobio sam samo tri prijave: jednu predugačku liniju u `logic/llm_adapters.py`, nekorišćenu promenljivu u `storage/redis.py:564` i prazan red sa belinama u `trainers.py:421`. Sa strožom granicom broj raste na 87, ali je 85 od njih samo predugačka linija.

Jedina prijava sa stvarnim značenjem je ona o nekorišćenoj promenljivoj. Rezultat poziva `add_texts` dodeljuje se promenljivoj koja se dalje ne koristi, dok se na skoro istom mestu, sedamdesetak linija iznad, taj isti rezultat koristi. Liči na propust nastao kopiranjem koda. Modul je van obima mog rada pa nisam dalje ispitivao.

Zaključujem da je projekat uredan po pitanju stila. Tri prijave na 10.400 linija je dobar rezultat i slabosti nisu tu.

5.5 mypy

Mypy proverava da li se anotacije tipova slažu sa onim što kod stvarno radi, i to bez pokretanja programa. ChatterBot koristi anotacije, ali ih ne proverava u svom CI-ju, pa nikada nisu bile provučene kroz alat.

Dobio sam 46 grešaka. Većina je šum kakav se očekuje kod projekta koji nikada nije prošao ovakvu proveru: nedostaju opisi tipova za biblioteke `yaml` i `dateutil`, a ima i promenljivih kojima se u toku funkcije menja tip. Dve prijave su, međutim, prave greške. Prva je na liniji `parsing.py:637`, gde se sabiraju `int` i `timedelta`, i ona zaista obara program. Druga je da funkcije u `parsing.py:564` i `response_selection.py:50` mogu da se izvrše do kraja i vrate `None`, iako u potpisu piše da vraćaju nešto drugo.

5.6 bandit

Bandit traži poznate bezbednosne obrasce, na primer korišćenje `eval`, slabe generatore slučajnih brojeva ili nesigurno raspakivanje arhiva. Prošao je kroz 7.674 linije i prijavio šest stvari: jednu visokog prioriteta, jednu srednjeg i četiri niskog.

Četiri nalaza niskog prioriteta odnose se na modul `random` i nisu problem, jer se on koristi za nasumičan izbor odgovora, gde bezbednost nije bitna. Nalaz srednjeg prioriteta je poziv `requests.get` bez vremenskog ograničenja: server koji ne odgovara može trajno da blokira preuzimanje korpusa. Nalaz visokog prioriteta obrađujem u sledećem poglavlju.

6 Pronađeni problemi

#	Gde	Problem	Nađeno pomoću
1	<code>parsing.py</code>	parser datuma puca umesto da vrati prazan rezultat, iz pet nezavisnih razloga	Hypothesis, mypy
2	<code>comparisons.py:183</code>	deljenje nulom za tekst od samih stop-reči	Hypothesis
3	<code>trainers.py:629</code>	zaštita od path traversal napada može da se zaobiđe	bandit
4	<code>comparisons.py:47</code>	mera sličnosti nije simetrična	Hypothesis
5	<code>constants.py:36</code>	semantička sličnost je neupotrebljiva sa podrazumevanim modelom	jedinični test
6	<code>conversation.py:92</code>	lokalno vreme je označeno kao UTC	jedinični test
7	<code>response_selection.py:50</code>	funkcija vraća <code>None</code> umesto obećanog tipa	mypy, jedinični test
8	<code>utils.py:26-27</code>	menja se rečnik koji je korisnik prosledio	jedinični test
9	<code>components.py</code> , <code>best_match.py:85</code>	bot ponavlja ono što je korisnik rekao	integracioni test

#	Gde	Problem	Nađeno pomoću
10	<code>trainers.py:75 i :310</code>	izvezena baza ne može da se uveze nazad	integracioni test

Tabela 3: Spisak pronađenih problema

Svaki nalaz je zapisan i kao test koji ga dokazuje. U nastavku detaljnije opisujem one najvažnije.

6.1 Parser datuma puca na običnim frazama

Funkcija `datetime_parsing` je javni deo biblioteke i opisana je u dokumentaciji. Kada u tekstu nema datuma, treba da vrati praznu listu. Umesto toga, na oko jedan procenat fraza sastavljenih od reči koje sama prepoznaje ona baca izuzetak. Pošto joj je ulaz korisnički tekst, to znači pad aplikacije koja je koristi.

Našao sam pet nezavisnih uzroka:

#	Ulaz	Uzrok
1a	"last month" u januaru	u <code>parsing.py:609</code> računa se <code>datetime(godina, mesec-1, dan)</code> , pa se u januaru dobija mesec 0
1b	"last month" 31. u mesecu	ista linija ne skraćuje dan na dužinu ciljnog meseca, pa 31. mart postaje "31. februar"; grana za "next month" to radi ispravno
1c	"end of the week"	u <code>parsing.py:636</code> uzima se <code>base_date.weekday()</code> , što je ceo broj, i zatim se sabira sa <code>timedelta</code>
1d	"29 hours"	izraz oblika "N sati" tumači se kao doba dana umesto kao trajanje, pa se pravi datum sa satom 29
1e	"end of the monday"	funkcija u <code>parsing.py:564</code> nema granu za "end of the", pa vrati <code>None</code> , a pozivalac na to dodaje <code>timedelta</code>

Tabela 4: Pet uzroka pada parsera datuma

Zanimljivo je da je uzroke 1c i 1e mypy prijavio statički, pre nego što sam uopšte pokušao da ih izazovem, a Hypothesis je nezavisno pronašao ulaze koji ih izazivaju. To je u ovom radu najjasnija potvrda da se te dve tehnike dopunjuju.

Ispravka bi podrazumevala da se prelaz preko granice meseca računa preko `calendar.monthrange`, da se sati sabiraju kao `timedelta`, da se doda grana koja nedostaje i da se ceo poziv zaštiti tako da neuspela obrada jednog fragmenta znači "ovo nije datum" umesto pada.

6.2 Zaštita od path traversal napada može da se zaobiđe

Bandit prijavljuje poziv `tarfile.extractall` u `trainers.py:640`. Na prvi pogled je to lažna uzbuna, jer projekat ima zaštitu: funkciju `safe_extract`, koja za svaki fajl iz

archive proverava da li bi završio unutar dozvoljenog direktorijuma.

```
def is_within_directory(directory, target):
    abs_directory = os.path.abspath(directory)
    abs_target = os.path.abspath(target)
    prefix = os.path.commonprefix([abs_directory, abs_target])
    return prefix == abs_directory
```

Problem je u funkciji `os.path.commonprefix`. Ona poredi stringove znak po znak, a ne putanje po delovima. Ako je dozvoljeni direktorijum `.../ubuntu_dialogs`, a fajl iz arhive ide u `.../ubuntu_dialogs_evil/`, zajednički prefiks ispada baš `.../ubuntu_dialogs`, pa provera prolazi iako je fajl izvan dozvoljenog direktorijuma.

Da bih to potvrdio, napisao sam skriptu koja pravi tar arhivu sa jednim takvim fajlom i poziva pravu metodu `extract`:

```
Rezultat extract()      : PROSAO bez izuzetka
Fajl van dozvoljenog dir.: True
ZAKLJUČAK: ZASTITA JE PROBIJENA
```

Rizik nije samo teorijski, jer se arhiva preuzima sa mreže preko HTTP-a i bez provere kontrolne sume. Reč je o ranjivosti klase CWE-22. Rešenje bi bilo koristiti `os.path.commonpath`, koja poredi delove putanje, ili, počev od Python-a 3.12, ugrađeni filter `tar.extractall(path, filter='data')`.

6.3 Bot ponavlja ono što je korisnik rekao

Ako se bot istrenira parom rečenica `['Good morning', 'Good morning to you too']`, on na pozdrav `'Good morning'` odgovara sa `'Good morning'`, dakle ponavlja korisnikov ulaz umesto naučenog odgovora.

Do toga dolazi ovako. Indeks za pretragu izbacuje stop-reči, pa obe rečenice dobiju isti indeks `ADJ:morning`. Adapter `BestMatch` u drugoj fazi traži sve rečenice sa istim indeksom kao pronađeni pogodak, da bi imao više varijanti odgovora. Zbog poklapanja indeksa u tu listu uđe i sam trenirani ulaz, jer filter izbacuje samo rečenice koje je izgovorio bot, a ne i one iz treninga. Na kraju se uzima prva iz liste, a to je korisnikov ulaz.

Greška se ne javlja za svaki par rečenica, što objašnjava zašto je do sada mogla da promakne.

6.4 Izvezeni podaci ne mogu da se uvezu

U opisu metode `export_for_training` piše da izvezeni fajl služi za treniranje drugih botova. Izvoz pravi objekat sa ključem `conversations` i listom parova pitanje–odgovor. Trener `JsonFileTrainer` čita ključ `conversation`, dakle u jednini, i očekuje listu rečenika sa poljem `text`. Uvoz zato puca sa greškom `KeyError`. Razlikuju se i ime ključa i oblik podataka, pa ono što je u dokumentaciji navedeno kao svrha funkcije zapravo ne radi.

7 Pokrivenost koda

Merenje je podešeno u fajlu `.coveragerc`. Najvažnije je isključenje modula `languages.py`: on ima 1.612 iskaza čistih podataka koje sam `import` “pokriva”, pa bi ukupna brojka sa njim skočila sa 53% na oko 78%, a da nijedan test ne bi bio bolji.

Merenje	Pokrivenost
Jedinični testovi	30%
Integracioni testovi	47%
Svi testovi zajedno	53%

Tabela 5: Pokrivenost koda po slojevima testiranja

Pokrivenost sam koristio kao mapu, a ne kao ocenu kvaliteta. Njena korist je u tome što pokazuje gde niko ne gleda. To se i potvrdilo: dva najozbiljnija nalaza su u modulima koji su pre mog rada imali najnižu pokrivenost, `parsing.py` sa 12% i `trainers.py` sa 0%.

8 Ograničenja

Slojevi za Redis, MongoDB i Django zahtevaju spoljne servise, pa ih nisam analizirao. Adaptere za velike jezičke modele nisam testirao sa stvarnim serverom. Nisam radio mutaciono testiranje, pa ne mogu da kažem koliko su moji testovi osetljivi na promene u kodu. Primetio sam da ponovljeni trening gomila duplikate u bazi, ali to nisam merio kao problem performansi. Nalaze nisam prijavio autoru projekta, pa je moguće da su neki u međuvremenu ispravljeni u novijim komitima.

9 Zaključak

ChatterBot je uredan projekat sa slabim rubnim slučajevima. Tri prijave alata flake8 na 10.400 linija koda i postojeći CI pokazuju da se projekat održava, ali deset nalaza iz ovog rada pokazuje da to održavanje nije stiglo do rubova.

Najviše grešaka sam našao tamo gde nije bilo testova. Moduli `parsing.py` i `trainers.py` imali su pokrivenost 12% odnosno 0%, a iz njih je došlo sedam od deset nalaza.

Statička i dinamička analiza su se stvarno dopunile. Dve greške u `parsing.py` mypy je prijavio bez pokretanja koda, a Hypothesis ih je nezavisno izazvao stvarnim ulazom. Ni jedna tehnika sama ne bi dala tu sigurnost, jer mypy prijavljuje i dosta šuma, a Hypothesis bez vođenja ne zna gde da traži.

Property-based testiranje mi je dalo najviše rezultata u odnosu na uloženo vreme, ali samo zato što sam ulaze pravio od reči koje parser prepoznaje. Sa potpuno nasumičnim tekstom ne bih našao ništa.

Naučio sam i da nalaz bezbednosnog alata ne treba odbaciti čim se vidi da zaštita postoji. Najozbiljniji nalaz u ovom radu nalazi se upravo u kodu koji je napisan da spreči tu ranjivost, ali koristi funkciju koja poredi stringove umesto putanja.

Na kraju, deo problema koje sam našao nisu padovi programa nego prekršena obećanja. Komparator baca izuzetak tamo gde srodne klase vraćaju nulu, funkcija vraća `None` iako u potpisu piše da vraća objekat, izvoz i uvoz koriste različite formate, a bot ume da ponovi ono što mu je korisnik rekao. Takve stvari se ne primete dok se ne napiše test koji naglas kaže šta se očekuje.

Prilog: struktura repozitorijuma

```
ChatterBot/      analizirani projekat (git submodule, komit feafc81)
tests/
  unit/          jedinicni testovi
  integration/   integracioni testovi
  property/      property-based testovi
tools/           po jedan folder za svaki alat: run.sh i results/
  unit-tests/    integration-tests/ hypothesis/ flake8/ mypy/ bandit/
scripts/
  priprema.sh    instalira zavisnosti
  pokreni_sve.sh pokrece svih sest alata
latex/          izvor ovog dokumenta
README.md       uputstvo i pregled rada
ProjectAnalysisReport.md detaljan izvestaj o analizi
```